

Assignment 2: Hybrid Images

Ella Moody and Ahmadreza Pourghodrat

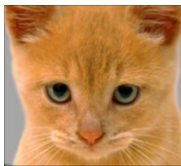
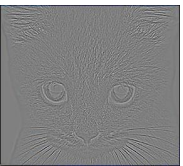
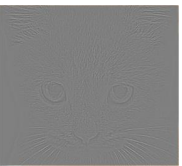
1. Image Filter

To make my code easier to debug in the beginning, I started with grayscale input images and debugged some steps and took care of the color dimension later which was pretty easy then. I firstly initialized an output of zeros with the same dimension as the input image. Then went through all the channels of the image which is an additional step to take care of the colored input images. Then for each channel, the code goes through each row of the image and for each row, it loops through each column of the image to choose the center pixel that should be used for the filter. The loops go through all the rows and columns regardless of the filter cell positions on the image because we'll take care of them falling out of the image later in the code. Before looping inside the filter, we set the sum value to 0 because the filter calculation is done for each pixel including and around the center overlapping with the filter. Then we go through each row of the filter and for each row we go through each column of the filter and get the filter value for that cell. We also need to know the image's pixel value on which the current filter cell is located at each step. To find that pixel's indexes, we calculate $\text{image_row} =$

$a - \text{floor}(f_width/2) + i - 1$ and $\text{image_col} = b - \text{floor}(f_height/2) + j - 1$. a and b are the current center pixel's row and column, while i and j are the current filter cell's row and column. Subtracting half of each filter dimension moves us toward the top-left corner of the neighborhood, and adding $i - 1$ and $j - 1$ moves us to the pixel corresponding to the current filter cell. This keeps the filter's center aligned with the image pixel at (a, b) .

If that pixel's indexes fall out of the image, we consider it as a padded black pixel and give it a value of 0. Otherwise, we just use those indexes to get the image value. Then we simply multiply the current pixel value by the current filter's cell value and add it to the sum and update the sum. Once we've gone through all the filter cells, we put the finalized sum for the center pixel to the output's corresponding pixel. Also to make sure that the calculations are done correctly, we cast the image and filter values to double.

Here are the outputs of the `my_imfilter` function for each filter test:

Identity	Small blur	Large blur	Sobel	Laplacian	High pass
					

I also added new lines to the test code to show the filter outputs from the matlab's built-in `imfilter` function and they looked exactly the same with `my_imfilter` outputs.

2. Hybrid Image

The actual code for this section is not very complicated. All five of the trials consisted of 1. Importing the image pair, 2. Creating a blurred image1 for the low frequency layer, 3. Creating a blurred image2 and subtracting that from the regular image2 to isolate the high frequencies, 4. Adding them together. The majority of the work came from both deciding what cutoff frequency (standard deviation in Gaussian) to use while using the `fspecial()` function to create the blur filters for image1 and image2. Additionally, I also had to figure out which image I wanted to be the low frequency image and which one to be the high frequency, since some images worked better as one or the other.

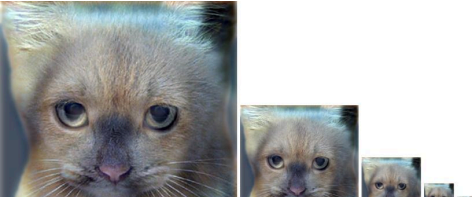
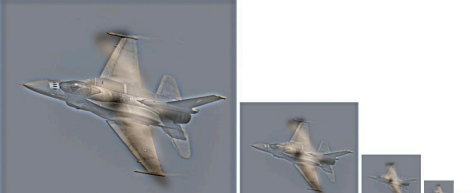
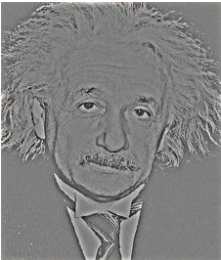
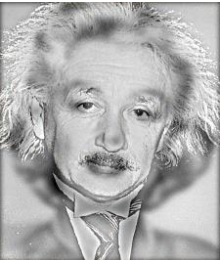
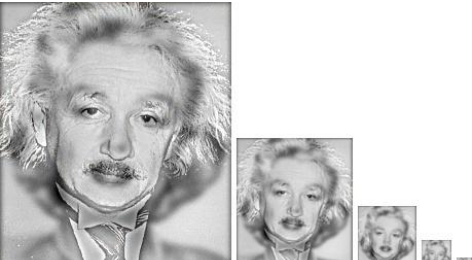
To figure out what cutoff frequency to use for both images and what order to put them in, it was largely trial and error with some thought process for how to change the numbers. After trying a lot of different combinations, I realized that the low frequency image always needs to be the one that can serve as a sort of “base” for the image. A really good example is the Submarine + Fish combination. Both the fish and the submarine have a similar body shape, but most of the fish’s recognizable traits are from finer details, and same for the cat in Dog + Cat. These images that are mostly distinguished by finer details serve a much better high frequency pick since those details can sort of “fade out” and leave just the general shape of the blurred image. If an image is mostly distinguishable by its general shape and NOT the finer details (that get lost from the blurring), it is a much better low frequency image. For the cutoff frequency, it was a little tricky for me to figure out since the high frequency logic is sort of inverted. In general, if I want the low frequency image to be less strong in an image, you should increase the cutoff frequency to have a greater blurring effect. However, because the high frequency image is just the regular image subtracted by the blurred image, if you want to make the high frequency image less strong in an image, you have to decrease the cutoff frequency. If the blurred image2 has a smaller cutoff frequency, it encompasses a larger range of values, so when you subtract it from the regular image2 it preserves less high frequencies.

I did notice that the most successful images were the grayscale ones or just ones that were similar colors. I think the least successful hybrid images were the Bicycle + Motorcycle and Bird + Plane, mostly because they are not colored similarly. I think if I were to turn all the images to grayscale first, I would have more success, especially since the Marilyn + Einstein one is the most effective hybrid image in my opinion.

With these findings, I determined both an ideal ordering and ideal cutoff frequency combination for all five pairings, the results of which are listed below.

	Low Frequency: Cutoff Frequency	High Frequency: Cutoff Frequency
Cat + Dog	7	7
Bicycle + Motorcycle	5	10

Bird + Plane	6	4
Marilyn + Einstein	3	3
Submarine + Fish	4	2

	Low Frequency	High Frequency	Hybrid	Hybrid Scaled
Dog + Cat				
Bicycle + Motorcycle				
Bird + Plane				
Marilyn + Einstein				
Submarine + Fish			